DOCUMENTAÇÃO DA API

Sistema Web de Gestão de Retiro

Versão 3.1 · Node.js (Express) + PostgreSQL · Março 2026

Componente	Tecnologia / Detalhe
Framework	Node.js + Express
Banco de Dados	PostgreSQL — 3 schemas (_meta | _sistema | retiro_AAAA)
Autenticação Admin	Sessão server-side com express-session + connect-pg-simple
Autenticação Jovem	Cookie HTTP-only com token UUID (sem login)
Formato de dados	JSON (request body e respostas)
Versionamento	Sem prefixo de versão na URL — API interna
Base URL	https://<domínio>/

1. Introdução

Esta documentação descreve todos os endpoints da API REST do Sistema Web de Gestão de Retiro, incluindo parâmetros, corpos de requisição, respostas e regras de autenticação. A API é consumida exclusivamente pelo frontend da própria aplicação web (não é pública).

1.1 Convenções

- Todos os corpos de requisição e respostas utilizam o formato JSON.
- Campos marcados como obrigatório devem estar presentes na requisição.
- Datas seguem o formato ISO 8601: YYYY-MM-DD ou YYYY-MM-DDTHH:MM:SSZ.
- UUIDs seguem o padrão RFC 4122 v4.
- Códigos HTTP de sucesso: 200 OK, 201 Created, 204 No Content.
- Erros retornam sempre um objeto JSON com o campo error (string).

1.2 Estrutura de Schemas

Toda requisição é resolvida em um ou mais dos três schemas PostgreSQL:

Schema	Responsabilidade	Acesso via
_meta	Eventos, colunas dinâmicas, usuários admin, audit log	Pool fixo meta.js
_sistema	Sessões de jovens, documentos (transversal)	Pool fixo sistema.js
retiro_AAAA	Inscrições, equipes, enums, pagamentos do evento	Pool dinâmico evento.js

1.3 Autenticação

A API possui dois mecanismos de autenticação independentes:

Cookie de Jovem (sessao_jovem)

Gerado automaticamente após inscrição bem-sucedida.

Token UUID v4, HTTP-only, Secure, SameSite=Strict.

Validado pelo middleware authJovem consultando _sistema.sessoes_jovens.

Não contém dados pessoais — apenas o token opaco.

Sessão Administrativa (express-session)

Login via POST /auth/login com login + senha.

Sessão armazenada no PostgreSQL via connect-pg-simple.

Perfis: admin | tesoureiro | lider.

Middleware authAdmin valida a sessão; middleware role.js verifica a função.

Líderes têm acesso restrito à própria congregação via middleware congregacao.js.

1.4 Códigos de Erro Padrão

Código HTTP	Significado	Causa comum
400 Bad Request	Dados inválidos ou ausentes	Campo obrigatório faltando, tipo incorreto
401 Unauthorized	Não autenticado	Cookie ausente/expirado, sessão inválida
403 Forbidden	Sem permissão	Perfil insuficiente, congregação diferente
404 Not Found	Recurso inexistente	ID ou slug não encontrado
409 Conflict	Conflito de estado	Inscrição duplicada, zero admins restantes
422 Unprocessable	Regra de negócio violada	Inscrições fechadas, equipe lotada
500 Internal Error	Erro interno	Falha de banco, erro inesperado

2. Autenticação

2.1 Login Administrativo

`POST /auth/login`

Autentica um usuário administrativo e cria sessão server-side.

Autenticação: Nenhuma (pública)

Body (JSON)

`login` string **obrigatório** — E-mail ou nome de usuário cadastrado em _meta.usuarios_admin

`senha` string **obrigatório** — Senha em texto plano — comparada com bcrypt/argon2

Respostas

`200 OK` { "usuario": { "id": 1, "nome": "...", "funcao": "admin|lider|tesoureiro", "congregacao": "..." } }

`401 Unauthorized` { "error": "Credenciais inválidas" }

`403 Forbidden` { "error": "Usuário inativo" }

⚠ **Observações**

- Se senha_inicial = true, a resposta inclui "trocar_senha": true e o cliente deve redirecionar para a tela de troca.
- Rate limit aplicado: máximo 10 tentativas por IP por minuto (express-rate-limit).

`POST /auth/logout`

Encerra a sessão administrativa e invalida o cookie de sessão.

Autenticação: Sessão administrativa

Respostas

`204 No Content` Sessão encerrada com sucesso

`PATCH /auth/senha`

Altera a senha do usuário autenticado. Obrigatório no primeiro login.

Autenticação: Sessão administrativa

Body (JSON)

`senha_atual` string **obrigatório** — Senha atual para confirmação

`nova_senha` string **obrigatório** — Nova senha (mínimo 8 caracteres)

Respostas

`200 OK` { "ok": true }

`400 Bad Request` { "error": "Senha atual incorreta" }

2.2 Recuperação de Acesso do Jovem

`POST /auth/recuperar`

Recupera a sessão do jovem quando o cookie foi perdido. Cria novo token em _sistema.sessoes_jovens.

Autenticação: Nenhuma (pública)

Body (JSON)

- `evento_slug` string opcional — Slug do evento. Se omitido, usa o evento padrão.
- `email` string opcional — E-mail cadastrado na inscrição, se utilizado no evento
- `documento` string opcional — CPF ou documento pessoal cadastrado, se utilizado no evento
- `codigo` string (UUID) opcional — Código único da inscrição

Respostas

- `200 OK` { "ok": true } — novo cookie sessao_jovem definido na resposta
- `400 Bad Request` { "error": "Informe ao menos um campo de identificação" }
- `404 Not Found` { "error": "Inscrição não encontrada" }

⚠ **Observações**

- Ao menos um dos campos email, documento ou código deve ser informado.
- Ao menos um dos campos email ou documento devem ser definidos para as colunas do evento.
- Rate limit: máximo 5 tentativas por IP por 15 minutos.

3. Rotas Públicas — Jovem

Acesso sem login. Identificação via cookie sessao_jovem.

3.1 Formulário de Inscrição

`GET /inscricao`

Retorna o schema de formulário do evento padrão ativo. Usado para renderizar o formulário de inscrição.

Autenticação: Nenhuma (pública)

Respostas

`200 OK` { "evento": { "id", "slug", "nome" }, "campos": [{ "nome", "label", "tipo", "obrigatorio", "opcoes"? }] }

`404 Not Found` { "error": "Nenhum evento padrão ativo com inscrições abertas" }

`GET /inscricao/:slug`

Retorna o schema de formulário de um evento específico identificado pelo slug.

Autenticação: Nenhuma (pública)

Parâmetros

`slug` string **obrigatório** — Slug do evento (ex: retiro_2026)

Respostas

`200 OK` { "evento": { ... }, "campos": [...] }

`404 Not Found` { "error": "Evento não encontrado" }

`POST /inscricao/:slug`

Cria nova inscrição no evento. Gera UUID, insere em retiro_AAAA.inscricoes e cria sessão em _sistema.sessoes_jovens.

Autenticação: Nenhuma — define cookie sessao_jovem na resposta

Parâmetros

`slug` string **obrigatório** — Slug do evento

Body (JSON)

`congregacao` string **obrigatório** — Congregação do jovem

`...campos_dinamicos` variado **obrigatório** — Campos definidos em _meta.colunas_schema para o evento

Respostas

`201 Created` { "codigo": "<UUID>", "evento_slug": "retiro_2026" } — cookie sessao_jovem é setado

`400 Bad Request` { "error": "Campo <nome> é obrigatório" }

`409 Conflict` { "error": "Já existe inscrição com este e-mail/documento" }

`422 Unprocessable` { "error": "Inscrições encerradas para este evento" }

⚠ **Observações**

- Schemas envolvidos: _meta (leitura de colunas), retiro_AAAA (INSERT inscrição), _sistema (INSERT sessão).
- O cookie é HTTP-only, Secure, SameSite=Strict e expira em 90 dias.

3.2 Minha Inscrição

`GET /minha-inscricao`

Retorna os dados da inscrição do jovem autenticado, com campos públicos, status de pagamento e equipe.

Autenticação: Cookie sessao_jovem

Respostas

`200 OK` { "inscricao": { "codigo_unico", "congregacao", "equipe_id", "cancelado", ...campos_dinamicos }, "pagamento": { "status": "pending|approved|manual", "valor", "modalidade" }, "equipe": { "id", "nome" } | null }

`401 Unauthorized` { "error": "Sem sessão ativa" }

⚠ **Observações**

- Apenas campos com visibilidade = 'public' são retornados. Campos restritos são omitidos.
- Schemas envolvidos:_sistema (recebe o slug), retiro_AAAA (SELECT inscrição),

`PUT /minha-inscricao`

Atualiza os dados cadastrais da própria inscrição. Disponível apenas enquanto as inscrições estiverem abertas.

Autenticação: Cookie sessao_jovem

Body (JSON)

`...campos_dinamicos` variado opcional — Campos editáveis definidos no schema do evento

Respostas

`200 OK` { "ok": true, "inscricao": { ...dados_atualizados } }

`422 Unprocessable` { "error": "Inscrições encerradas — edição não permitida" }

⚠ **Observações**

- Não é possível alterar congregação ou código único após a inscrição.

3.3 Equipe

GET /minha-equipe

Retorna a equipe atribuída ao jovem e a lista de membros (apenas nome, sem dados sensíveis).

Autenticação: Cookie sessao_jovem

Respostas

200 OK { "equipe": { "id", "nome", "capacidade" }, "membros": [{ "nome" }] }

200 OK { "equipe": null } — jovem ainda não atribuído a nenhuma equipe

3.4 Documentos

POST /documentos

Realiza upload de um documento para a inscrição do jovem. Armazena em _sistema.documentos.

Autenticação: Cookie sessao_jovem

Body (JSON)

`arquivo` multipart/form-data **obrigatório** — Arquivo do documento (PDF, JPG, PNG — máx. 5 MB)

`nome` string **obrigatório** — Nome/descrição do documento (ex: RG, Contrato)

Respostas

201 Created { "documento": { "id", "nome", "enviado_em", "enviado_por": "jovem" } }

400 Bad Request { "error": "Tipo de arquivo não permitido" }

⚠ **Observações**

- Utiliza multer para o upload. O caminho do arquivo é salvo em _sistema.documentos.caminho.

3.5 Pagamento

POST /pagamento/iniciar

Cria uma preferência de pagamento no gateway (ex: Mercado Pago) e retorna a URL de checkout.

Autenticação: Cookie sessao_jovem

Body (JSON)

`modalidade` string **obrigatório** — Modalidade de pagamento: diaria | integral | (enums do evento)

`valor` number **obrigatório** — Valor em reais

Respostas

200 OK { "checkout_url": "https://...", "gateway_id": "..." }

422 Unprocessable { "error": "Pagamento já aprovado para esta inscrição" }

`POST /pagamento/webhook`

Recebe notificações do gateway de pagamento. Atualiza status em retiro_AAAA.pagamentos e registra em audit_log.

Autenticação: Assinatura HMAC do gateway (Header X-Signature)

Body (JSON)

`payload_raw` object **obrigatório** — Payload bruto enviado pelo gateway

Respostas

`200 OK` { "ok": true }

`400 Bad Request` { "error": "Assinatura inválida" }

⚠ **Observações**

- A assinatura HMAC é validada antes de qualquer processamento.
- Em caso de status approved, a inscrição é marcada como confirmada.
- Idempotente: gateway_id duplicado é ignorado silenciosamente.

4. Rotas do Líder

Requer sessão administrativa com funcao = lider.

`GET /admin/lider/inscricoes`

Lista inscrições da congregação do líder autenticado. Aplica automaticamente o filtro de congregação.

Autenticação: Sessão admin — funcao: lider

Parâmetros

- `status` string opcional — Filtro: todos | pagos | pendentes (padrão: todos)
- `pagina` number opcional — Paginação (padrão: 1)
- `por_pagina` number opcional — Itens por página (padrão: 50, máx: 200)

Respostas

`200 OK` { "total": 42, "pagina": 1, "inscricoes": [{ "codigo_unico", "nome_completo", "status_pagamento", ...campos_visiveis }] }

⚠ Observações

- Apenas campos com visibilidade = 'public' ou visivel_para contendo 'lider' são retornados.

`POST /admin/lider/inscricoes`

Inscreve manualmente um jovem na congregação do líder. Fluxo idêntico ao do jovem, congregação preenchida automaticamente.

Autenticação: Sessão admin — funcao: lider

Body (JSON)

- `...campos_dinamicos` variado **obrigatório** — Mesmos campos do formulário público do evento

Respostas

- `201 Created` { "inscricao": { "codigo_unico", ... } }
- `409 Conflict` { "error": "Já existe inscrição com este documento/e-mail" }

`DELETE /admin/lider/inscricoes/:codigo`

Remove (cancela) uma inscrição da congregação do líder. Registra motivo e grava em audit_log.

Autenticação: Sessão admin — funcao: lider

Parâmetros

- `codigo` string (UUID) **obrigatório** — Código único da inscrição

Body (JSON)

- `motivo` string **obrigatório** — Motivo do cancelamento (registrado em inscricoes.motivo_cancelamento)

Respostas

`200 OK` { "ok": true }

403 Forbidden { "error": "Inscrição pertence a outra congregação" }
404 Not Found { "error": "Inscrição não encontrada" }

GET /admin/lider/documentos/:codigo

Lista documentos enviados para uma inscrição da congregação do líder.

Autenticação: Sessão admin — funcao: lider

Parâmetros

`codigo` string (UUID) **obrigatório** — Código único da inscrição

Respostas

200 OK { "documentos": [{ "id", "nome", "enviado_por", "enviado_em", "caminho" }] }

POST /admin/lider/documentos/:codigo

Adiciona um documento a uma inscrição da congregação. enviado_por = 'lider'.

Autenticação: Sessão admin — funcao: lider

Parâmetros

`codigo` string (UUID) **obrigatório** — Código único da inscrição

Body (JSON)

`arquivo` multipart/form-data **obrigatório** — Arquivo (PDF, JPG, PNG — máx. 5 MB)
`nome` string **obrigatório** — Nome/descrição do documento

Respostas

201 Created { "documento": { "id", "nome", "enviado_em", "enviado_por": "lider" } }

5. Rotas do Tesoureiro

Requer sessão administrativa com funcao = tesoureiro.

GET /admin/tesoureiro/pagamentos

Lista todos os pagamentos do evento ativo com filtros opcionais.

Autenticação: Sessão admin — funcao: tesoureiro | admin

Parâmetros

- `congregacao` string opcional — Filtra por congregação
- `status` string opcional — pending | approved | rejected | manual
- `tipo` string opcional — online | manual
- `pagina` number opcional — Paginação (padrão: 1)

Respostas

`200 OK` { "total": 120, "pagamentos": [{ "id", "inscricao_codigo", "nome_jovem", "congregacao", "tipo", "modalidade", "status", "valor", "data_pagamento", "forma_pagamento", "observacao" }] }

⚠ **Observações**

- nome_jovem é obtido via JOIN com inscrições usando apenas campos com visibilidade pública.

GET /admin/tesoureiro/pendentes

Retorna resumo de pagamentos pendentes agrupados por congregação.

Autenticação: Sessão admin — funcao: tesoureiro | admin

Respostas

`200 OK` { "congregacoes": [{ "nome": "Congregação A", "pendentes": 5, "total_esperado": 750.00 }] }

PATCH /admin/tesoureiro/pagamentos/:id/marcar

Marca um pagamento como pago manualmente. Define marcado_manual = true e registra dados do pagamento.

Autenticação: Sessão admin — funcao: tesoureiro | admin

Parâmetros

- `id` integer **obrigatório** — ID do pagamento em retiro_AAAA.pagamentos

Body (JSON)

- `data_pagamento` string (date) **obrigatório** — Data em que o pagamento foi recebido (YYYY-MM-DD)
- `forma_pagamento` string **obrigatório** — pix | dinheiro | transferencia | cheque
- `valor` number **obrigatório** — Valor recebido em reais
- `observacao` string opcional — Observação livre

Respostas

`200 OK` { "ok": true, "pagamento": { ...dados_atualizados } }

`409 Conflict` { "error": "Pagamento já aprovado via gateway" }

⚠ **Observações**

- AuditService registra a ação em _meta.audit_log com snapshot antes/depois.

`GET /admin/tesoureiro/gateway`

Lista transações recebidas do gateway de pagamento com seu status original.

Autenticação: Sessão admin — funcao: tesoureiro | admin

Parâmetros

`status` string opcional — pending | approved | rejected

`de` string (date) opcional — Data inicial (YYYY-MM-DD)

`ate` string (date) opcional — Data final (YYYY-MM-DD)

Respostas

`200 OK` { "transacoes": [{ "gateway_id", "status", "valor", "criado_em", "payload_raw" }] }

6. Rotas do Administrador

Requer sessão administrativa com funcao = admin.

6.1 Dashboard e Inscrições

GET /admin/dashboard

Retorna visão consolidada com totais de inscrições, pagamentos e distribuição por equipe e congregação.

Autenticação: Sessão admin — funcao: admin

Respostas

200 OK { "total_inscritos": 80, "pagos": 60, "pendentes": 20, "por_congregacao": [...], "por_equipe": [...], "por_tipo_inscricao": [...] }

GET /admin/inscricoes

Lista todas as inscrições do evento ativo com suporte a filtros dinâmicos e paginação.

Autenticação: Sessão admin — funcao: admin

Parâmetros

- `congregacao` string opcional — Filtra por congregação
- `equipe_id` integer opcional — Filtra por equipe
- `status_pagamento` string opcional — pending | approved | manual
- `cancelado` boolean opcional — true | false (padrão: false)
- `pagina` number opcional — Paginação (padrão: 1)
- `por_pagina` number opcional — Itens por página (padrão: 100)

Respostas

200 OK { "total": 200, "inscricoes": [{ ...todos_os_campos, "equipe", "status_pagamento" }] }

PATCH /admin/inscricoes/:codigo

Edita dados de qualquer inscrição. Registra alteração em audit_log.

Autenticação: Sessão admin — funcao: admin

Parâmetros

- `codigo` string (UUID) **obrigatório** — Código único da inscrição

Body (JSON)

- `...campos` variado opcional — Qualquer campo dinâmico ou congregacao

Respostas

200 OK { "ok": true, "inscricao": { ...dados_atualizados } }

⚠ **Observações**

- Snapshot antes/depois gravado em _meta.audit_log.

DELETE /admin/inscricoes/:codigo

Cancela uma inscrição. Define cancelado = true e registra motivo.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`codigo` string (UUID) **obrigatório** — Código único da inscrição

Body (JSON)

`motivo` string **obrigatório** — Motivo do cancelamento

Respostas

`200 OK` { "ok": true }

`404 Not Found` { "error": "Inscrição não encontrada" }

6.2 Eventos

GET /admin/eventos

Lista todos os eventos cadastrados.

Autenticação: Sessão admin — funcao: admin

Respostas

`200 OK` { "eventos": [{ "id", "slug", "nome", "ativo", "padrao", "inscricoes_abertas", "inscricoes_abertas_de", "inscricoes_abertas_ate" }] }

POST /admin/eventos

Cria novo evento e provisiona automaticamente o schema PostgreSQL com as tabelas padrão.

Autenticação: Sessão admin — funcao: admin

Body (JSON)

`slug` string **obrigatório** — Identificador único (ex: retiro_2027) — usado como nome do schema

`nome` string **obrigatório** — Nome de exibição do evento

`descricao` string opcional — Descrição opcional

`inscricoes_abertas_de` string (datetime) opcional — Data/hora de abertura das inscrições

`inscricoes_abertas_ate` string (datetime) opcional — Data/hora de encerramento das inscrições

Respostas

`201 Created` { "evento": { "id", "slug", "nome", ... } }

`409 Conflict` { "error": "Slug já existe" }

⚠ **Observações**

- EventoService executa CREATE SCHEMA <slug> + CREATE TABLE inscricoes, equipes, pagamentos.
- Slug é validado como identificador SQL seguro (somente letras, números e underscore).

`PATCH /admin/eventos/:id`

Edita metadados do evento (nome, descrição, datas, status ativo/inativo).

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do evento

Body (JSON)

`nome` string opcional — Novo nome
`descricao` string opcional — Nova descrição
`ativo` boolean opcional — Ativa ou desativa o evento
`inscricoes_abertas` boolean opcional — Abre ou fecha inscrições manualmente
`inscricoes_abertas_de` string (datetime) opcional — Data de abertura
`inscricoes_abertas_ate` string (datetime) opcional — Data de encerramento

Respostas

`200 OK` { "ok": true, "evento": { ...dados_atualizados } }

`PATCH /admin/eventos/:id/padrao`

Define o evento como evento padrão. Automaticamente remove o flag padrão de todos os outros eventos.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do evento a promover como padrão

Respostas

`200 OK` { "ok": true }
`422 Unprocessable` { "error": "Evento inativo não pode ser definido como padrão" }

⚠ **Observações**

- EventoService garante que apenas um evento tenha padrao = true. Ação registrada em audit_log.

6.3 Colunas / Schema Dinâmico

GET /admin/eventos/:id/colunas

Lista as colunas dinâmicas definidas para o formulário do evento.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do evento

Respostas

`200 OK` { "colunas": [{ "id", "nome", "label", "tipo", "obrigatorio", "visibilidade", "visible_to", "ordem", "opcoes"? }] }

POST /admin/eventos/:id/colunas

Adiciona uma nova coluna ao schema do evento. SchemaService executa ALTER TABLE para adicionar a coluna na tabela de inscrições.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do evento

Body (JSON)

`nome` string **obrigatório** — Nome técnico da coluna (ex: tamanho_camiseta_enum_tamanhos)

`label` string **obrigatório** — Rótulo de exibição (ex: Tamanho da Camiseta)

`tipo` string **obrigatório** — text | number | date | boolean | email | enum

`obrigatorio` boolean opcional — Padrão: true

`visibilidade` string opcional — public | restricted (padrão: public)

`visivel_para` array<string> opcional — Ex: ["lider", "admin"] — usado quando visibilidade = restricted

`ordem` integer opcional — Ordem de exibição no formulário

Respostas

`201 Created` { "coluna": { "id", "nome", "label", ... } }

`409 Conflict` { "error": "Coluna com este nome já existe no evento" }

`409 Conflict` { "error": "Coluna com este label já existe no evento" }

⚠ Observações

- Para tipo = enum, o nome deve seguir o padrão <descricao>_enum_<tabela>.
- SchemaService cria a tabela enum_<tabela> automaticamente se não existir.

DELETE /admin/eventos/:id/colunas/:colId

Remove uma coluna do schema do evento. Executa ALTER TABLE DROP COLUMN.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do evento

`colId` integer **obrigatório** — ID da coluna em _meta.colunas_schema

Respostas

`200 OK` { "ok": true }
`404 Not Found` { "error": "Evento não encontrado" }
`404 Not Found` { "error": "Coluna não encontrada" }
`422 Unprocessable` { "error": "Coluna possui dados — use force=true para forçar remoção" }

⚠ **Observações**

- Ação irreversível. Dados da coluna são perdidos permanentemente.

6.4 Equipes

`GET /admin/equipes`

Lista equipes do evento ativo com contagem de membros.

Autenticação: Sessão admin — funcao: admin | lider

Respostas

`200 OK` { "equipes": [{ "id", "nome", "capacidade", "membros": 12 }] }

`POST /admin/equipes`

Cria nova equipe para o evento ativo.

Autenticação: Sessão admin — funcao: admin

Body (JSON)

`nome` string **obrigatório** — Nome da equipe
`cor` string **obrigatório** — Cor da equipe

Respostas

`201 Created` { "equipe": { "id", "nome", "capacidade" } }
`400 Bad Request` { "error": "Campos cor ou nome faltando" }
`400 Bad Request` { "error": "Campo cor fora do formato hexadecimal" }

`PATCH /admin/equipes/:id`

Edita nome ou capacidade de uma equipe.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID da equipe

Body (JSON)

`nome` string opcional — Novo nome
`cor` integer opcional — Nova cor

Respostas

`200 OK` { "ok": true, "equipe": { ...dados_atualizados } }

`400 Bad Request` { "error": "Campo cor fora do formato hexadecimal" }

`DELETE /admin/equipes/:id`

Exclui uma equipe. Jovens atribuídos têm equipe_id definido como null.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID da equipe

Respostas

`200 OK` { "ok": true, "jovens_desvinculados": 8 }

`404 Not Found` { "error": "Equipe não encontrada" }

`POST /admin/equipes/sortear`

Distribui aleatoriamente todos os jovens sem equipe entre as equipes disponíveis.

Autenticação: Sessão admin — funcao: admin

Body (JSON)

`evento_slug` string opcional — Slug do evento (padrão: evento padrão ativo)

Respostas

`200 OK` { "distribuidos": 24, "sem_vaga": 0, "resultado": [{ "equipe_id", "equipe_nome", "novos_membros": 6 }] }

⚠ **Observações**

- EquipeService usa algoritmo de distribuição aleatória com balanceamento.

`PATCH /admin/equipes/atribuir`

Atribui ou transfere manualmente um jovem para uma equipe específica.

Autenticação: Sessão admin — funcao: admin

Body (JSON)

`inscricao_codigo` string (UUID) **obrigatório** — Código único da inscrição

`equipe_id` integer | null **obrigatório** — ID da equipe destino. null remove o jovem de qualquer equipe.

Respostas

`200 OK` { "ok": true }

`404 Not Found` { "error": "Equipe ou inscrição não encontrados" }

6.5 Usuários Administrativos

GET /admin/usuarios

Lista todos os usuários administrativos cadastrados.

Autenticação: Sessão admin — funcao: admin

Respostas

`200 OK` { "usuarios": [{ "id", "nome", "login", "funcao", "congregacao", "ativo", "criado_em" }] }

⚠ **Observações**

- O campo senha_hash nunca é retornado pela API.

POST /admin/usuarios

Cria novo usuário administrativo. Senha armazenada com bcrypt (salt ≥ 12) ou Argon2.

Autenticação: Sessão admin — funcao: admin

Body (JSON)

`nome` string **obrigatório** — Nome completo
`login` string **obrigatório** — E-mail ou usuário único para login
`senha` string **obrigatório** — Senha inicial (usuário obrigado a trocar no 1º acesso)
`funcao` string **obrigatório** — admin | tesoureiro | lider
`congregacao` string opcional — Obrigatório quando funcao = lider

Respostas

`201 Created` { "usuario": { "id", "nome", "login", "funcao", "congregacao" } }
`409 Conflict` { "error": "Login já cadastrado" }
`400 Bad Request` { "error": "congregação obrigatória para lider" }
`400 Bad Request` { "error": "congregação não encontrada" }

⚠ **Observações**

- O usuário é criado com senha_inicial = true. Ação registrada em audit_log.

PATCH /admin/usuarios/:id

Edita dados de um usuário: nome, login, função, congregação ou redefine a senha.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do usuário

Body (JSON)

`nome` string opcional — Novo nome
`login` string opcional — Novo login
`funcao` string opcional — Nova função
`congregacao` string opcional — Nova congregação (obrigatório para lider)

`nova_senha` string opcional — Se informado, redefine a senha e seta senha_inicial = true

Respostas

`200 OK` { "ok": true, "usuario": { ...dados_atualizados } }

`409 Conflict` { "error": "Alteração resultaria em zero administradores ativos" }

`404 Not Found` { "error": "usuário não encontrado" }

PATCH /admin/usuarios/:id/status

Ativa ou desativa um usuário administrativo.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`id` integer **obrigatório** — ID do usuário

Body (JSON)

`ativo` boolean **obrigatório** — true para ativar, false para desativar

Respostas

`200 OK` { "ok": true }

`404 Not Found` { "error": "usuário não encontrado" }

`409 Conflict` { "error": "Não é possível desativar o único administrador ativo" }

⚠ **Observações**

- Bloqueio de zero admins: o sistema impede desativação se restar apenas 1 admin ativo.

6.6 Audit Log

GET /admin/audit

Consulta o log de auditoria de ações administrativas.

Autenticação: Sessão admin — funcao: admin

Parâmetros

`usuario_id` integer opcional — Filtra por usuário autor da ação

`entidade` string opcional — inscricoes | equipes | pagamentos | usuarios_admin | eventos

`acao` string opcional — Ex: INSCRICAO_EDITADA, PAGAMENTO_MARCADO, USUARIO_CRIADO

`de` string (date) opcional — Data inicial

`ate` string (date) opcional — Data final

`pagina` number opcional — Paginação (padrão: 1)

Respostas

`200 OK` { "total": 1200, "logs": [{ "id", "evento_slug", "usuario_id", "usuario_nome", "acao", "entidade", "entidade_id", "payload", "ip", "criado_em" }] }

⚠ **Observações**

- O campo payload contém snapshot JSONB com { antes: {...}, depois: {...} } para alterações.

7. Exemplos de Requisição e Resposta

7.1 Criar Inscrição

Requisição:

```
POST /inscricao/retiro_2026
Content-Type: application/json

{
  "congregacao": "Santa Fé",
  "nome_completo": "Maria Silva",
  "email": "maria@email.com",
  "documento_pessoal": "123.456.789-00",
  "tamanho_camiseta_enum_tamanhos": 2,
  "tipo_inscricao_enum_tipos": 1
}
```

Resposta — 201 Created:

```
{
  "codigo": "a3f2c1d0-7e4b-4a8f-b1c2-9d0e5f6a7b8c",
  "evento_slug": "retiro_2026"
}

// Header de resposta:
// Set-Cookie: sessao_jovem=<uuid>; HttpOnly; Secure; SameSite=Strict
```

7.2 Marcar Pagamento Manual

Requisição:

```
PATCH /admin/tesoureiro/pagamentos/42/marcar
Content-Type: application/json
```

```
{
  "data_pagamento": "2026-03-10",
  "forma_pagamento": "pix",
  "valor": 150.00,
  "observacao": "PIX recebido às 10h30"
}
```

Resposta — 200 OK:

```
{
  "ok": true,
  "pagamento": {
    "id": 42,
    "status": "manual",
    "marcado_manual": true,
    "data_pagamento": "2026-03-10",
    "forma_pagamento": "pix",
    "valor": 150.00,
    "marcado_por": 3
  }
}
```

7.3 Criar Evento

Requisição:

```
POST /admin/eventos
Content-Type: application/json

{
  "slug": "retiro_2027",
  "nome": "Retiro Anual 2027",
  "descricao": "Edição 2027 do retiro anual de jovens",
  "inscricoes_abertas_de": "2027-01-15T08:00:00Z",
  "inscricoes_abertas_ate": "2027-03-01T23:59:59Z"
}
```

Resposta — 201 Created:

```
{
```

```
  "evento": {
    "id": 3,
    "slug": "retiro_2027",
    "nome": "Retiro Anual 2027",
    "ativo": true,
    "padrao": false,
    "inscricoes_abertas": false,
    "inscricoes_abertas_de": "2027-01-15T08:00:00Z",
    "inscricoes_abertas_ate": "2027-03-01T23:59:59Z"
  }
}

// Schema PostgreSQL "retiro_2027" criado automaticamente.
```

8. Matriz de Endpoints

Método	Rota	Perfil mínimo	Schemas
GET	/inscricao	Público	_meta
GET	/inscricao/:slug	Público	_meta
POST	/inscricao/:slug	Público	_meta, retiro_AAAA, _sistema
GET	/minha-inscricao	Cookie jovem	_sistema, retiro_AAAA, _meta
PUT	/minha-inscricao	Cookie jovem	_sistema, retiro_AAAA
GET	/minha-equipe	Cookie jovem	_sistema, retiro_AAAA
POST	/documentos	Cookie jovem	_sistema
POST	/pagamento/iniciar	Cookie jovem	_sistema, retiro_AAAA
POST	/pagamento/webhook	HMAC gateway	retiro_AAAA, _meta
POST	/auth/login	Público	_meta
POST	/auth/logout	Sessão admin	_meta
PATCH	/auth/senha	Sessão admin	_meta
POST	/auth/recuperar	Público	retiro_AAAA, _sistema
GET	/admin/lider/inscricoes	Líder	retiro_AAAA, _meta
POST	/admin/lider/inscricoes	Líder	retiro_AAAA, _sistema, _meta
DELETE	/admin/lider/inscricoes/:codigo	Líder	retiro_AAAA, _meta
GET	/admin/lider/documentos/:codigo	Líder	_sistema
POST	/admin/lider/documentos/:codigo	Líder	_sistema
GET	/admin/tesoureiro/pagamentos	Tesoureiro	retiro_AAAA
GET	/admin/tesoureiro/pendentes	Tesoureiro	retiro_AAAA
PATCH	/admin/tesoureiro/pagamentos/:id/marcar	Tesoureiro	retiro_AAAA, _meta
GET	/admin/tesoureiro/gateway	Tesoureiro	retiro_AAAA
GET	/admin/dashboard	Admin	retiro_AAAA, _meta
GET	/admin/inscricoes	Admin	retiro_AAAA, _meta
PATCH	/admin/inscricoes/:codigo	Admin	retiro_AAAA, _meta
DELETE	/admin/inscricoes/:codigo	Admin	retiro_AAAA, _meta
GET	/admin/eventos	Admin	_meta

Método	Rota	Perfil mínimo	Schemas
POST	/admin/eventos	Admin	_meta
PATCH	/admin/eventos/:id	Admin	_meta
PATCH	/admin/eventos/:id/padrao	Admin	_meta
GET	/admin/eventos/:id/colunas	Admin	_meta
POST	/admin/eventos/:id/colunas	Admin	_meta, retiro_AAAA
DELETE	/admin/eventos/:id/colunas/:colId	Admin	_meta, retiro_AAAA
GET	/admin/equipes	Admin | Líder	retiro_AAAA
POST	/admin/equipes	Admin	retiro_AAAA
PATCH	/admin/equipes/:id	Admin	retiro_AAAA
DELETE	/admin/equipes/:id	Admin	retiro_AAAA
POST	/admin/equipes/sortear	Admin	retiro_AAAA
PATCH	/admin/equipes/atribuir	Admin	retiro_AAAA
GET	/admin/usuarios	Admin	_meta
POST	/admin/usuarios	Admin	_meta
PATCH	/admin/usuarios/:id	Admin	_meta
PATCH	/admin/usuarios/:id/status	Admin	_meta
GET	/admin/audit	Admin	_meta